Alyssa Nursadrina
24/540219/TK/59925

ASD
ASSIGNMENT #1

#1 Snippet A

i.) Count T(n) !

Count T(n) !	Assignment	cost	Execute	Total
int sum = 0 ;	1	1	1	1
for (int i=0; i<n ; i++) {outer loop)				
↳ int = 0	1	1	1	1
↳ i<n	1	1	n+1	n+1
↳ i++	2	2	n	2n
for (int j=0 ; j<i ; j++) (inner loop)				
↳ int j = 0	1	1	n	n
↳ j<i	1	1	$\frac{n(n+1)}{2}$	$\frac{n^2+n}{2}$
↳ j++	2	2	$\frac{n(n-1)}{2}$	n^2-n
Sum += arr [i] + arr[j] ;	5	5	$\frac{n(n-1)}{2}$	$\frac{5}{2}(n^2-n)$

T(n)

- $n^2 = \frac{1}{2}n^2 + n^2 + \frac{5}{2}n^2$
  $= 4n^2$
- $n = n + 2n + n + \frac{1}{2}n - n - \frac{5}{2}n$
  $= n$
- constant = 3

↳ $\boxed{T(n) = 4n^2 + n + 3}$

ii) Big O class

↳ $O(n^2)$ → constant of highest orde

iii) Best / worst case

↳ Equal (none best / worst), number of iteration in nested loop only based on pure n value and did not have break or evaluation logic.

Snippet B

i.) Count T(n) !

	cost	Execute	Total
int lo = 0 , hi = n-1 , target = 42 ; bool found = false ;	5 (4 assignment, 1 arithmetic)	1	5
while (lo <= hi)	1 (comparison)	a+1	a+1
int mid = lo + (hi-lo)/2 ;	4 (3 arithmetic, 1 assign)	a	4a
if (arr[mid] == target)	2 (array, comparation)	a	2a
else if (arr[mid] < target)	2 (array, comparation)	a	2a
else {hi = mid-1 ; }	2 (arithmetic, assign)	a	2a
T(n)			

- $5 + (a+1) + 4a + 2a + 2a + 2a$
  $= 11a + 6$
- $a = \log_2 n + 1$

↳ $\boxed{T(n) = 11 \log_2 n + 17}$

Dipindai dengan CamScanner

ii) Big O Class

↳ $O(\log n)$

iii) Best /Worst Case

↳ Best case : Jika target lgsg ditemukan pd iterasi pertama (loop 1 → break)

Maka, kompleksitas : $O(1)$

↳ Worst case : Jika tidak di dalam array atau di ujung, loop berjalan hingga berukuran 1. Iterasi sebanyak $\log_2 n+1$ kali.

Maka, kompleksitas : $O(\log n)$

Snippet C

i) Count $T(n)$

	Cost	Execute	Total
int count = 0 ;	1 (assign)	1	1
int i = 0	1 (assign)	1	1
i < n	1 (Comparison)	n+1	n+1
i ++	2 (arithmetic, assign)	n	2n
int j = i+1 ;	2 (arithmetic, assign)	n	2n
j < n	1 (comparison)	$n^2+n/2$	$n^2+n/2$
j ++	2 (arithmetic, assign)	$n^2-n/2$	n^2-n
int k=0 ;	1 (assign)	L_2	$1/2\, n^2 - n/2$
k < n	1 (comparison)	$L_2(M+1)$	$L_2(M+1)$
k + = 2	2 (arithmetic, assign)	$L_2 \times M$	$2L_2 M$
If (arr[i]! = arr[j] && arr[k] > 0)	6 (3 array, 2 compare, 1 logic)	$L_2 \times M$	$6L_2 M$
Count ++	2 (arithmetic, assign)	$L_2 \times M$	$2L_2 M$
$T(n)$	$= 1 + (3n+2) + (3/2\, n^2 + 3/2\, n) + (3/4\, n^3 + 1/4\, n^2 - n) + (3/2\, n^3 - 3/2\, n^2) + (1/2\, n^3 - 1/2\, n^2)$		
	$= 11/4\, n^3 - 1/4\, n^2 + 7/2\, n + 3$		
	$= \boxed{\frac{11}{4} n^3 - \frac{1}{4} n^2 + \frac{7}{2} n + 3}$		

ii) Big O Class

↳ $O(n^3)$

iii) Best /worst case

None (same)

↳ 3 loop has static structure and always run exactly $O(n^3)$ Whatever the array is.

3B

① Complexity Companion Table

Operation	Best Case	Worst Case	Average Case
Linear Search (unsorted)	$O(1)$	$O(n)$	$O(n)$
Linear Search (sorted, early exit)	$O(1)$	$O(n)$	$O(n)$
Binary Search (sorted)	$O(1)$	$O(\log n)$	$O(\log n)$

Dipindai dengan CamScanner

2.) Explanation

ⓐ Linear Search (unsorted)

- Best case : O(1) → elements target are found in first index array and the loop stops when first checked.
- Worst case : O(n) → elements target not found in array or at last index and the algorithm forced to checked all one by one.
- Average case : O(n) → Average elements target will found after searching on half array ($n/2$)

ⓑ Linear Search (sorted, early exit)

- Best case : O(1) → Elements target found in early array / the elements on first array already bigger than target.
- Worst case : O(n) → Elements target are bigger than all elements in array or at the end, so the early exit not triggered and forced to checked n element.
- Average case : O(n) → The program can stopped early when it already found bigger number, but the algorithm still needs to scanned half of the array (similar O(n)).

ⓒ Binary Search (sorted)

- Best case : O(n) → Elements target is at the middle value (mid) on spliting iteration when first executed.
- Worst case : O(log n) → Elements are not found on the first / last extreme array so it is need to splitted until its become 1.
- Average case : O(log n) → Most of the elements array are on below level of decision tree. The average iteration needed ≈ $\log_2 n$.

3) Reasoning

The Binary Search became absolutely more efficient than linear search when size of array minimum n=3.

↳ When array n=3, BS (2 iteration) in mathematical doing, fewer iteration, compare to LS (3 iteration).

	LS	BS	
n = 1	1	$\lceil \log_2 1 \rceil + 1 = 1$	1
n = 2	1+1	2	$\lceil \log_2 2 \rceil + 1 = 2$	2
n = 3	1+1+1	3	$\lceil \log_2 3 \rceil + 1 = 2$	3

#4

A)

Call #	Value Pushed	Size After	Capacity After	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	3
4	40	4	4	1
5	50	5	8	5
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	9
10	100	10	16	1
11	110	11	16	1
12	120	12	16	1

B) ① Sum all values in Cost

$T_{12} = 1+1+3+1+5+1+1+1+9+1+1+1$

$T_{12} = 26$

② Average cost per call

$\frac{26}{12} \approx 2,167$ operation / call.

SiDU

Dipindai dengan CamScanner

③ Total copy work across n pushbacks (Geometric series)

↳ formula and evaluation : The copy occurs at 2, 4, 6, 8, ... size for n push bonds

Total : $2 + 4 + 8 + \dots + 2^k = \boxed{\sum_{i=1}^{k} 2^i}$

(where 2^k is the largest power of 2 strictly less than n)

↳ The sum of geometric series is exactly $2^{k+1} - 2$, because : $2^k < n \rightarrow 2^{k+1} \leq 2n$

There for total copy is strictly less than $\boxed{2n}$

④ What is this tell us

↳ Total cost for n operations, (normal n appends) + (total copy work $\leq 2n$) $\leq 3n$.

↳ The amortised cost per call is the total cost divided by n : $\leq 3/n \leq 3$

↳ This tell us that the amortised cost per call is $\boxed{O(1) \text{ (constant time)}}$

c) Growth factor Comparison.

① Redo the trace (fixed k = 4)

Call #	Value Pushed	Size After	Capacity After	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	6	3
4	40	4	6	1
5	50	5	6	1
6	60	6	6	1
7	70	7	10	7
8	80	8	10	1
9	90	9	10	1
10	100	10	10	1
11	110	11	14	11
12	120	12	14	1

② compare total cost :

↳ from part B = 26

↳ fixed = 30

↳ So, doubling strategy are more efficient.

③ General Derivation. T(n) amortised cost :

- Doubling strategy

↳ Total cost :

$T(n) = n \text{ (appends)} + O(n) \text{ (copies)}$

$= O(n)$

↳ Amortized cost :

$\frac{O(n)}{n} = O(1)$

- fixed Increment Strategy (k)

↳ Resize happens at size c, c+k, c+2k, ..

↳ Number of resize for n approx n/k

↳ Total copy work B : Sum of arithmetic series :

$$\sum_{i=1}^{n/k} (ik) = k \cdot \frac{n/k\,(n/k+1)}{2} \approx \frac{n^2}{2k} = O(n^2)$$

↳ Total cost $T(n) = n \text{ (appends)} + O(n^2) \text{ (copies)} = O(n^2)$

↳ Amortized cost = $\frac{O(n^2)}{n} = O(n)$.

CS Dipindai dengan CamScanner